ELEC50014 Software Systems
Final Examination

Time: 2 hours Marks: 60

Question 1 [20]

(a) Solution:

i. **FALSE.** A process cannot transition directly from the new state to waiting; it must first run before it can block.

ii. **FALSE.** Even if a system call does not block, the process may still be preempted by timer interrupts while executing in kernel mode.

iii. **FALSE.** Memory accesses may trigger page faults, which trap into the kernel even without I/O or system calls.

iv. **FALSE.** Shared memory mappings may use different virtual addresses in each process while referring to the same physical pages.

v. **TRUE.** Code pages are typically read-only and can be shared between parent and child processes for their entire lifetime.

vi. **TRUE.** Reducing the quantum increases scheduling overhead due to more frequent context switching, so the average response time is not guaranteed to decrease.

vii. **FALSE.** Even the highest-priority queue uses time slicing, so a job can be preempted rather than running to completion.

viii. **FALSE.** CFS allocates CPU time proportionally to process weights, not equally among all ready processes.

(b) Solution:

i. **FCFS:**

$$\text{ATT} = \frac{(7-0)+(9-2)+(14-4)}{3} = \frac{24}{3} = 8$$

Non-preemptive SJF:

Same execution order as FCFS, hence

$$\text{ATT} = \frac{24}{3} = 8$$

STCF:

finish times: $B = 4,\ A = 9,\ C = 14$ $\quad \text{ATT} = \frac{(9-0)+(4-2)+(14-4)}{3} = \frac{21}{3} = 7$

ii. RR (quantum = 1): ATT increases relative to each of the above. RR time-slices blindly and does not prioritise short jobs, delaying completion times, while STCF is optimal for minimizing ATT.

Question 2 [20]

(a) Solution:

(i) This follows the standard pattern of semaphore usage.

```
void* access_db_reliably(void* arg)
{
    sem_wait(&sem);
    access_db(arg);
    sem_post(&sem);
    return NULL;
}
```

(ii) As there are N resources (slots) to give away, the semaphore needs to be initialized to N.

```
    sem_init(&sem, 0, N);
```

(b) Solution:

Not Possible. With mutexes only, a thread can check and update a shared slot counter safely, but if no slots are available it has no mechanism to sleep and later be woken when a slot is released. A mutex provides mutual exclusion, not waiting/wakeup.

(c) Solution:

One possible implementation:

```
void acquire_slot()
{
    int expected = permits;

    while (expected <= 0 ||
           compare_and_swap(&permits,
                            expected,
                            expected - 1) != expected)
           {
                expected = permits;
           }
}

void release_slot()
{
    fetch_and_add(&permits);
}
```

(d) Solution:

Incorrect. The problem occurs when N threads each successfully execute the first call to `acquire_slot`, consuming all N slots. Each thread then busy-waits on its second `acquire_slot` and no progress is made.

(e) Solution:

Both Option 1 and Option 2 are viable.
Option 1: A condition variable / mutex is used to allow threads to wait when there are no available slots. When a slot becomes available, all waiting threads may be woken with a broadcast, and the one whose turn it is next is allowed to take a slot (the turn logic is implemented at user level).
Option 2: A ticket-based mechanism is implemented to give threads FIFO turns, ensuring that slots are assigned in arrival order and fairness is enforced.

Question 3 [20]

(a) Solution:

i. Page size = 4 KB = 2^{12} bytes. Page offset bits = 12. Virtual page number bits = $42 - 12 = 30$.

ii. Physical memory = 4 GB = 2^{32} bytes. Number of page frames = $2^{32}/2^{12} = 2^{20}$. Page frame number width = 20 bits.

iii. Number of virtual pages = 2^{30}. Linear page table size = 2^{30} entries $\times 4$ bytes = 2^{32} bytes. Pages needed = $2^{32}/2^{12} = 2^{20}$ pages.

iv. Max. number of entries per page table page = $2^{12}/2^2 = 2^{10}$. Therefore, number of bits used to index per level = 10. Total levels needed = $30/10 = 3$.

v. The process can address up to 2^{27} bytes. Therefore, number of virtual pages used = $2^{27}/2^{12} = 2^{15}$ pages.

Each level covers 2^{10} entries.

Level 3 pages needed = $2^{15}/2^{10} = 2^5 = 32$. Level 2 pages needed = 1 (only 32 entries needed, fits in one page). Level 1 pages needed = 1 (only one entry needed, fits in one page).

(b) Solution:

Two main reasons for this design choice relate to system correctness and performance, respectively.

(1) Possible circular dependency during address translation: when a TLB miss occurs, the hardware page-table walk must access the page tables in memory. If a page table page were itself swapped out (present bit 0), this would trigger a page fault and transfer control to the kernel. The kernel would then need to access page tables to service the fault, potentially causing further page faults. This creates a circular dependency and can prevent the system from making progress.

(2) System performance: address translation is required for every memory reference (instruction fetches and data accesses). If page table pages could be swapped

out, their frequent use would cause repeated page faults and disk I/O, leading to extremely high latency and severe overall system slowdown.

(c) Solution:

i. Larger virtual page sizes: Not always. Larger pages increase TLB reach, but if the program's working set already fits in the TLB, the number of misses may remain unchanged.

ii. Smaller virtual address space size: Not always. TLB misses depend on the working set and access pattern rather than the maximum representable address range. A smaller address space does not necessarily reduce misses.

iii. Increased TLB size: Not always. TLB misses depend on the memory access pattern. If the working set of pages already fits within the smaller TLB, increasing the TLB size will not change the miss count. Hence fewer misses are not guaranteed.